Université Sultan Moulay Slimane
Ecole Supérieure de Technologie
- Beni Mellal -

STRUCTURES DE DONNÉES

Chapitre 4 : Listes Doublement Chainées

Filière : DUT GI
Semestre : S3
Année Universitaire : 2026/2027

Pr. M. OUTANOUTE



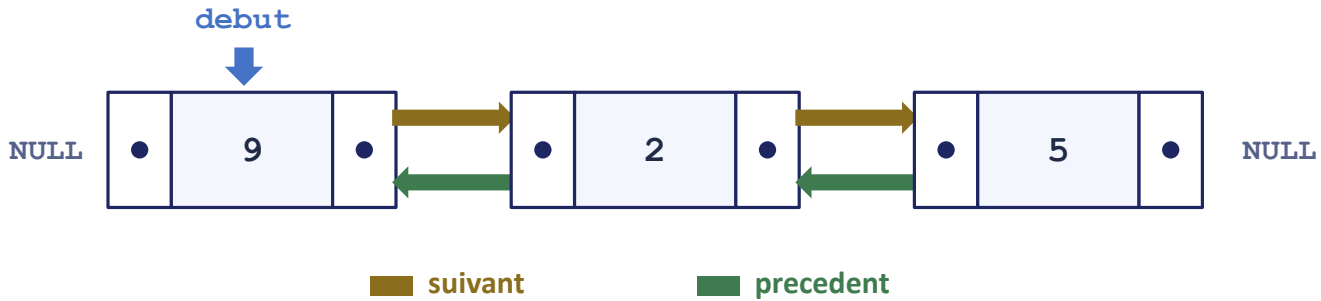
Rappel : le piège du Chapitre 2

Dans une liste simplement chaînée (chapitre 2), chaque nœud ne connaît que son ***suivant***. Pour supprimer un élément donné, il fallait suivre ***precedent*** manuellement, dans une variable à part, pendant tout le parcours.

Et si chaque nœud connaissait déjà, lui-même, celui qui le précède ?

L'idée : un second pointeur, dans le nœud

```c
typedef struct noeud {
    int valeur;
    struct noeud *suivant;
    struct noeud *precedent;
} Noeud;
```



__debut__ reste la seule variable globale — pas de __fin__ dans ce chapitre (ce gain appartient au Chapitre 3).



Ce que ça change concrètement

Le gain de ce chapitre :

Supprimer un élément donné n'a plus besoin de deux pointeurs suivis manuellement — un seul parcours suffit, le nœud trouvé connaît déjà son voisin de gauche.

Bonus :

Un parcours arrière devient possible — **de la fin vers le début** — impossible au Chapitre 2 (LSC).

Le fil narratif : registre amélioré

Le registre des inscrits du Chapitre 2 (LSC), repris ici.

La radiation d'un étudiant, en particulier, en profite directement : plus simple à écrire et à comprendre.

Déclaration

```
typedef struct noeud {
    int valeur;
    struct noeud *suivant;
    struct noeud *precedent;
} Noeud;

Noeud *debut = NULL;
```

debut : variable globale, comme au Chapitre 2 (LSC).

Parcours avant, et parcours arrière

```
void afficherListe(void) {
    Noeud *courant = debut;
    while (courant != NULL)
           { printf("%d ", courant->valeur); courant = courant->suivant; }
}
```

1) Si la liste est vide, rien à afficher.
2) Sinon, avancer d'abord avec suivant jusqu'au dernier nœud — pas de repère fin dans ce chapitre.
3) Depuis ce dernier nœud, reculer avec precedent jusqu'à NULL.

```
void afficherListeEnvers(void) {
    Noeud *courant = debut;
    if (courant == NULL) { printf("\n"); return; }
    while (courant->suivant != NULL) { courant = courant->suivant; }
    while (courant != NULL)
          { printf("%d ", courant->valeur); courant = courant->precedent; }
}
```

Insertion en tête

1) Créer un ***nouveau*** nœud, lui donner sa ***valeur***.
2) Faire pointer *son* ***suivant*** vers l'*ancien* ***debut***, et *son* ***precedent*** vers *NULL (il sera le premier)*.
3) Si la liste n'était pas vide, faire pointer le ***precedent*** de l'*ancien* ***debut*** vers le ***nouveau*** nœud.
4) ***debut*** devient le ***nouveau*** nœud.

```
void insererEnTete(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    nouveau->suivant = debut;
    nouveau->precedent = NULL;
    if (debut != NULL) { debut->precedent = nouveau; }
    debut = nouveau;
}
```

Insertion en fin

1) Créer un ***nouveau*** nœud, *son* ***suivant*** à *NULL* (il sera le dernier).
2) Si la liste est vide, *son* ***precedent*** est *NULL* aussi, et il devient ***debut***.
3) Sinon, parcourir jusqu'au *dernier nœud* (celui dont ***suivant*** vaut *NULL*).
4) Faire pointer le ***suivant*** de ce *dernier nœud* vers le ***nouveau***, et *son* ***precedent*** vers ce *dernier nœud*.

```
void insererEnFin(int valeur) {
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    nouveau->suivant = NULL;
    if (debut == NULL) { nouveau->precedent = NULL; debut = nouveau; return; }
    Noeud *courant = debut;
    while (courant->suivant != NULL) { courant = courant->suivant; }
    courant->suivant = nouveau;
    nouveau->precedent = courant;
}
```

Insertion après une valeur donnée

1) Parcourir la liste jusqu'à trouver le nœud contenant *la valeur* ***apres***, ou atteindre *la fin*.
2) Si non trouvé, rien à faire.
3) Créer un ***nouveau*** *nœud*, faire pointer *son* ***suivant*** vers ***courant->suivant***, et *son* ***precedent*** vers ***courant***.
4) Si ***courant->suivant*** n'est pas *NULL*, faire pointer *son* ***precedent*** vers le ***nouveau*** *nœud*.
5) Faire pointer le ***suivant*** *de* ***courant*** vers le ***nouveau*** *nœud*.

```
void insererApresValeur(int apres, int valeur) {
    Noeud *courant = debut;
    while (courant != NULL && courant->valeur != apres) { courant = courant->suivant; }
    if (courant == NULL) { return; }
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    nouveau->suivant = courant->suivant;
    nouveau->precedent = courant;
    if (courant->suivant != NULL) { courant->suivant->precedent = nouveau; }
    courant->suivant = nouveau;
}
```

Insertion avant une valeur donnée

Là où ***precedent*** change vraiment la donne :
au Chapitre 2 (LSC), insérer avant un nœud exigeait la même vigilance qu'une suppression.

1) Parcourir la liste jusqu'à trouver le nœud contenant la *valeur* ***avant***, ou atteindre *la fin*.
2) Si non trouvé, rien à faire.
3) Si le nœud trouvé est ***debut***, cas particulier — traiter comme une ***insertion en tête***.
4) Sinon, créer un ***nouveau*** *nœud*, *son* ***suivant*** vers ***courant***, *son* ***precedent*** vers ***courant->precedent***.
5) Faire pointer le ***suivant*** *de* ***courant->precedent*** vers le ***nouveau*** *nœud*.
6) Faire pointer le ***precedent*** *de* ***courant*** vers le ***nouveau*** *nœud*.

insererAvantValeur — le code

```
void insererAvantValeur(int avant, int valeur) {
    Noeud *courant = debut;
    while (courant != NULL && courant->valeur != avant)
         { courant = courant->suivant; }
    if (courant == NULL) { return; }
    if (courant == debut) { insererEnTete(valeur); return; }
    Noeud *nouveau = malloc(sizeof(Noeud));
    nouveau->valeur = valeur;
    nouveau->suivant = courant;
    nouveau->precedent = courant->precedent;
    courant->precedent->suivant = nouveau;
    courant->precedent = nouveau;
}
```

Le même principe que ***supprimerValeur*** : ***precedent*** simplifie aussi bien l'insertion que la suppression.

Suppression en tête

1) Si la liste est vide, rien à faire.
2) Mémoriser le *nœud* ***debut*** *actuel*.
3) Faire avancer ***debut*** vers ***debut->suivant***.
4) Si la liste n'est pas devenue vide, remettre le ***precedent*** *du nouveau* ***debut*** à NULL.
5) Libérer l'*ancien nœud*.

```
void supprimerEnTete(void) {
    if (debut == NULL) { return; }
    Noeud *ancienDebut = debut;
    debut = debut->suivant;
    if (debut != NULL) { debut->precedent = NULL; }
    free(ancienDebut);
}
```



LE point fort : supprimer un élément donné

Le gain le plus important de ce chapitre.

1) Parcourir la liste avec un seul pointeur, ***courant***, jusqu'à trouver la ***valeur*** ou atteindre *la fin*.
2) Si non trouvé, rien à faire.
3) Si le nœud trouvé est ***debut***, cas particulier — traiter comme une ***suppression en tête***.
4) Sinon, faire pointer le ***suivant*** *de* ***courant->precedent*** vers ***courant->suivant***.
5) Si ***courant->suivant*** n'est pas *NULL*, faire pointer *son* ***precedent*** vers ***courant->precedent***.
6) Libérer ***courant***.

supprimerValeur — le code

```
void supprimerValeur(int valeur) {
    Noeud *courant = debut;
    while (courant != NULL && courant->valeur != valeur)
        { courant = courant->suivant; }
    if (courant == NULL) { return; }
    if (courant == debut) { supprimerEnTete(); return; }
    courant->precedent->suivant = courant->suivant;
    if (courant->suivant != NULL)
        { courant->suivant->precedent = courant->precedent; }
    free(courant);
}
```

Comparer avec le Chapitre 2 (LSC) :

un seul ***courant*** suffit — ***courant->precedent*** est déjà disponible, porté par le nœud lui-même.

Recherche, et détruire la liste

```
int rechercher(int valeur) {
    Noeud *courant = debut;
    while (courant != NULL) {
        if (courant->valeur == valeur) { return 1; }
        courant = courant->suivant;
    }
    return 0;
}
```

```
void detruireListe(void) {
    while (debut != NULL) { supprimerEnTete(); }
}
```

Identiques au Chapitre 2 (LSC) — ***precedent*** n'intervient dans aucune des deux.

Récapitulatif

- Déclaration : Noeud avec ***suivant*** ET ***precedent*** ; ***debut*** seul, variable globale.
- Parcours arrière : atteindre le *dernier nœud* via ***suivant***, puis reculer via ***precedent***.
- Insertion en tête/en fin : mettre à jour ***precedent*** en plus de ***suivant***.
- Insertion après une valeur : identique au Chapitre 2 (LSC), ***precedent*** n'y change rien.
- Insertion avant une valeur : là où ***precedent*** change tout — *un seul* ***courant*** suffit.
- Le point fort, en miroir : ***supprimerValeur*** ET ***insererAvantValeur*** n'ont besoin que de ***courant***.
- Cas particulier : si ***courant == debut***, pas de ***precedent*** — on retombe sur les *fonctions en tête*.
- ***rechercher*** et ***detruireListe*** : identiques au Chapitre 2 (LSC).

Exercice d’application

Le registre des inscrits d'une faculté est représenté par une ***liste doublement chaînée***, ***debut*** étant une variable globale.

Un étudiant est caractérisé par un numéro, un nom et un prénom.

1) Écrire ***inscrireEtudiant*** (en fin) et ***radierEtudiant*** (par numéro).
2) Dérouler l'inscription de 3 étudiants (101, 102, 103), puis la radiation de 102 et 101.
3) Comparer, en une phrase, ***radierEtudiant*** à la version du Chapitre 2 (LSC).
4) Pourquoi la radiation du premier étudiant reste-t-elle un cas particulier, même dans ce chapitre ?

1) Le code : déclarations

```
typedef struct {
   int numero;
   char nom[50];
   char prenom[50];
} Etudiant;

typedef struct noeudEtudiant {
   Etudiant donnees;
   struct noeudEtudiant *suivant;
   struct noeudEtudiant *precedent;
} NoeudEtudiant;

NoeudEtudiant *debut = NULL;
```

1) Le code : fonction *inscrireEtudiant(Etudiant);*

```
void inscrireEtudiant(Etudiant E) {
    NoeudEtudiant *nouveau = malloc(sizeof(NoeudEtudiant));
    nouveau->donnees = E;
    nouveau->suivant = NULL;
    if (debut == NULL) {
        nouveau->precedent = NULL;
        debut = nouveau;
        return;
    }
    NoeudEtudiant *courant = debut;
    while (courant->suivant != NULL) courant = courant->suivant;
    courant->suivant = nouveau;
    nouveau->precedent = courant;
}
```

1) Le code : fonction *radierEtudiant(int);*

```
void radierEtudiant(int num) {
    NoeudEtudiant *courant = debut;
    while (courant != NULL && courant->donnees.numero != num)
        courant = courant->suivant;
    if (courant == NULL) return;
    if (courant == debut) {
        debut = debut->suivant;
        if (debut != NULL) debut->precedent = NULL;
        free(courant); return;
    }
    courant->precedent->suivant = courant->suivant;
    if (courant->suivant != NULL)
        courant->suivant->precedent = courant->precedent;
    free(courant);
}
```

2) La trace

```
Etudiant E1={101,"AAA","aaa"}, E2={102,"BBB","bbb"}, E3={103,"CCC","ccc"};
```

Étape	Action	État de la liste
1	`inscrireEtudiant(E1)`	`[101,AAA,aaa]`
2	`inscrireEtudiant(E2)`	`[101,AAA,aaa] ↔ [102,BBB,bbb]`
3	`inscrireEtudiant(E3)`	`[101,AAA,aaa] ↔ [102,BBB,bbb] ↔ [103,CCC,ccc]`
4	`radierEtudiant(102)`	`[101,AAA,aaa] ↔ [103,CCC,ccc]`
5	`radierEtudiant(101)`	`[103,CCC,ccc]`

102 était au milieu : ***courant*** l'a trouvé en un seul parcours, et les deux voisins se reconnectent directement.

101 était au début : on avance ***debut***, et comme la liste n'est pas devenue vide, on met ***debut->precedent*** à NULL.

3) La comparaison

Contrairement au Chapitre 2 (LSC), où ***radierEtudiant*** devait suivre deux pointeurs (***precedent*** et ***courant***) pendant tout le parcours, ici un seul pointeur suffit :

courant>precedent est déjà porté par le nœud trouvé, sans variable supplémentaire à maintenir.

4) L'explication

Supprimer ***debut*** change la variable globale elle-même, pas seulement les pointeurs d'un nœud voisin — un cas que ***courant->precedent*** ne peut pas couvrir, puisque ***debut*** n'a justement pas de *précédent*. C'est pour cette raison que ***supprimerValeur*** (et ***radierEtudiant***) traitent ce cas à part, en réutilisant ***supprimerEnTete***.

Exercice

Écrire une fonction ***inverserListe*** qui inverse l'ordre des éléments de la liste doublement chainée (globale) **sans créer aucun nouveau nœud** — uniquement en échangeant, dans chaque nœud, ***suivant*** et ***precedent***.

Utiliser les déclarations et la fonction ***inscrireEtudiant*** de l'exercice d'application pour tester la fonction ***inverserListe*** en affichant - avant et après l'inversion - la liste contenant les trois étudiants E1, E2 et E3.

Indices pour inverser la liste :

- *Un seul parcours suffit.*
- *Dans chaque nœud visité, échanger simplement les deux pointeurs — ce qui était **suivant** devient **precedent**, et inversement — avant d'avancer vers ce qui était l'ancien **suivant**.*
- *Ne pas oublier de réaffecter **debut** à la toute fin : c'est l'ancien dernier nœud.*